§6.H Telemetry Credential Leak Audit — 2026-06-23

Audit type: READ-ONLY, zero-bluff
Scope: Every non-fatal / warning telemetry call site added this session, plus all redaction infrastructure those call sites depend on.
Verdict: CLEAN — no credential, token, cookie, key, pepper, handoff-key, or Auth-Token value can reach Firebase Crashlytics, structured logs, or the observability webhook through any of the audited paths.
Notable (non-leak, documented): `ApiHttpException.message` carries the raw request URL including query-string search terms. See §7.

---

1. Architectural invariant (primary protection)

The `Lava-Auth` / handoff key is set as an **HTTP request header** via `withAuth()` in `ApiBackedTrackerClient.kt:126–137`. It is never placed in a URL query parameter. This means no URL-stripping mechanism can fail to protect it — it never enters a URL in the first place. All query-string protection below is defence-in-depth for search-term privacy, not for auth-key safety.

---

2. Android — per-call-site verdicts

2.1 `ApiBackedTrackerClient.kt` — `ApiHttpException` construction

File: `core/tracker/client/src/main/kotlin/lava/tracker/client/ApiBackedTrackerClient.kt`

Field	Construction	Credential risk
`statusCode`	HTTP response status integer	None
`httpMethod`	HTTP verb string (`"GET"`)	None
`requestUrl`		

Field	Construction	Credential risk
	`stripQueryForTelemetry(url)` **L276, L291, L311**	CLEAN — strips everything after `?`
`responseSnippet`	`redactAndTruncate(resp.body?.string())` **L273, L288, L308**	CLEAN — truncated to 200 chars + regex redaction
`message` (throwable)	`"API request failed: HTTP ${resp.code} for $url"` **L279, L293, L314**	See §7 — raw URL with search terms, NOT auth values

`stripQueryForTelemetry` (L330-336): `val idx = url.indexOf('?'); if (idx >= 0) url.substring(0, idx) else url`
`redactAndTruncate` (L348-352): `.take(200)` then `.replace(Regex("(?i)(token|key|cookie|auth|bearer|password|secret)([=:\\s]+)\\S+"), "$1$2[REDACTED]")`

Auth key placement: `withAuth()` at L126-137 sets `header(authFieldName, authKey)` and `header(AUTH_TOKEN_HEADER, sessionToken)`. Headers are never part of the URL string passed to `stripQueryForTelemetry`. CLEAN.

2.2 `SearchResultViewModel.kt` — `recordProviderFailure`

File: `feature/search_result/src/main/kotlin/lava/search/result/SearchResultViewModel.kt`

Context map values at L177-207:

Key	Value source	Credential risk
`FEATURE`	literal `"search"`	None
`OPERATION`	literal `"streamMultiSearch"`	None
`SCREEN`	literal `"search_result"`	None
`PROVIDER`		None

Key	Value source	Credential risk
	`event.providerId` — tracker-id string like `"rutracker"`	
`ERROR_MESSAGE`	`event.reason` — user-facing reason string	None
`HTTP_STATUS`	`cause.statusCode.toString()`	None
`REQUEST_URL`	`cause.requestUrl` — already query-stripped via `ApiHttpException`	CLEAN
`HTTP_METHOD`	`cause.httpMethod` — HTTP verb	None
`BASE_URL_HOST`	`cause.requestUrl.substringBefore("/")` — host portion only, no query	CLEAN

The throwable `cause` (with its message containing search-term URL) goes to `analytics.recordNonFatal(cause, context)` at L211. The message is captured by Crashlytics `recordException(throwable)`. See §7 for the search-terms-in-URL note. No auth values present.

2.3 `ApiKeyClient.kt` — `recordWarning` on SecurityException / Exception

File: `app/src/main/kotlin/digital/vasic/lava/client/handoff/ApiKeyClient.kt`

Two `recordWarning` calls (L66-73 and L80-87):

SecurityException path (L66-73):

```
message: "API key provider permission denied for $authority"
context: FEATURE="handoff", OPERATION="read_api_key",
         ERROR_MESSAGE=(e.message ?: "SecurityException")
```

`$authority` is the ContentProvider authority string (e.g. `digital.vasic.lava.api.keyprovider`) — a package-path string, not the key value. `e.message` is the OS-level `SecurityException` message ("Permission denied" class). **The key value `cursor.getString(keyIdx)` is read only in the success branch (L57) and is NEVER passed to any telemetry call.** CLEAN.

Generic Exception path (L80-87):

```
message: "API key provider read failed for $authority"
context: FEATURE="handoff", OPERATION="read_api_key",
         ERROR_MESSAGE=(e.message ?: "unknown")
```

Same analysis. `e.message` is a provider-absent / I/O exception message — does not contain the key value, which is only obtained on success. CLEAN.

Source-code comment at L94: `// §6.H: this tag is attached only to NON-SECRET diagnostic messages; the access key value is NEVER logged.` This is accurate and verified by inspection.

2.4 `OnboardingViewModel.kt` — all `recordNonFatal` / `recordWarning` call sites

File: `feature/onboarding/src/main/kotlin/lava/onboarding/OnboardingViewModel.kt`

L354-358 (api_probe_failed):

```
context: ERROR="api_probe_failed"
```

Only one attr. The exception `e` is a network/timeout exception from the connectivity probe — does not contain auth keys (the probe uses `connectionService.isReachable()`, not a keyed HTTP call). CLEAN.

L415-424 (`withLocalApiKeyIfMissing`, apiKeyReader exception):

```
message: "API key reader failed for local endpoint: ${e.message}"
context: FEATURE="handoff", OPERATION="read_api_key_for_endpoint",
         ERROR_MESSAGE=(e.message ?: "unknown")
```

The message includes `e.message` (exception description) — never the key value, which is obtained only in the success branch (`apiKeyReader?.invoke()` return value at L425). The comment at L415: `// §6.H: authority/message only, never the key value.` CLEAN.

L659-664 (connection_test_failed):

```
context: PROVIDER=currentId, ERROR="connection_test_failed"
```

`currentId` is a tracker-id string. `e` is passed as the throwable. The connection test exception comes from `sdk.login()` / `sdk.checkAuthorized()` — these return structured `AuthState` results; the exception message does not contain credential values (which are passed to the SDK as separate parameters, not embedded in exception messages). CLEAN.

L815-820 (`on_device_api_key_read_failed`):

```
message: "on_device_api_key_read_failed"
context: ERROR_MESSAGE=(e.message ?: "unknown")
```

Comment at L819: `// no-telemetry: key value is never included — §6.H`. The `apiKeyReader?.invoke()` return value (the key) is only in the non-exception path (L828:

`val endpoint = Endpoint.GoApi(host = host, port = port, key = apiKey)`). Exception branch records only the exception message, not the key. CLEAN.

L884-887 and L894-897 (provider_catalog_fetch_failed):

`context: ERROR="provider_catalog_fetch_failed"`

Single attr, no URL or key. The fetch failure is from `useCase(apiBaseUrl, goApi.key)` where the key is passed as a function argument — it appears nowhere in the exception message. CLEAN.

2.5 `ApiEngineService.kt` — FGS `recordNonFatal`

File: `api-app/src/main/kotlin/lava/api/app/service/ApiEngineService.kt`

```
analytics.recordNonFatal(
    e,                                                    //
        ForegroundServiceStartNotAllowedException
    mapOf(
        FEATURE to "api_engine",
        MODULE  to "ApiEngineService",
        OPERATION to "onStartCommand",
        ERROR   to "foreground_start_not_allowed",
    ),
)
```

(L136-144)

The exception `e` is `ForegroundServiceStartNotAllowedException` — an OS-level Android exception with a fixed message ("ForegroundServiceStartNotAllowedException"). No credentials are available at `onStartCommand()` entry — the service holds a reference to `ApiEngineController` and `AnalyticsTracker` only. Comment at L135: `// §6.H: no credentials in context (none are available here).` CLEAN.

2.6 `FirebaseAnalyticsTracker.kt` — the sink

File: `core/analytics-firebase/src/main/kotlin/lava/analytics/firebase/FirebaseAnalyticsTracker.kt`

The sink behaviour:

- `recordNonFatal` (L51-65): calls `c.setCustomKey(key, value.take(1024))` for each context entry, then `c.recordException(throwable)`. The context values are whatever the **call site** provides; the sink does NOT redact. Redaction responsibility is fully on the call sites.

- `recordWarning` (L96-104): calls `c.log("WARN: ${message.take(1024)} ctx=$context")` — logs all context as a breadcrumb string.

This means the audit's correctness rests entirely on the call sites NOT passing secret values. The per-call-site analysis above confirms they do not. CLEAN.

---

3. Backend (lava-api-go) — per-call-site verdicts

3.1 `internal/observability/nonfatal.go` — redaction infrastructure

File: `lava-api-go/internal/observability/nonfatal.go`

`sensitiveAttrPatterns` (L98-108):

```
[]string{"password", "token", "secret", "api_key", "apikey",
         "cookie", "authorization", "hmac", "pepper"}
```

`redactIfSensitive` (L296-304): `strings.Contains(strings.ToLower(key), pat)` — if any pattern appears as a **substring** of the attribute key name, the value is replaced with `"<redacted>"`.

`webhookForward` (L184-235): builds a `safe` map applying `redactIfSensitive(k, v)` to **ALL** attrs before JSON-marshalling and POSTing. CLEAN.

Pattern coverage analysis for Lava-Auth:
The Lava-Auth field name is read from the env-var `LAVA_AUTH_FIELD_NAME` (§6.R — not hardcoded). Its value is an operator-chosen string (e.g. `"X-Lava-Auth"`). The `sensitiveAttrPatterns` list contains `"authorization"` (matches `Authorization` header variants) but NOT `"auth"` as a standalone substring. However, examining every Go call site that passes attrs to `RecordNonFatal` / `RecordWarning` (§3.2 and §3.3 below) confirms: **no call site passes the Lava-Auth field name or its value as an attr key or value.** The pattern gap does not result in a leak because the data is never present in attrs. CLEAN.

3.2 `internal/handlers/v1/handlers.go` — `writeProviderError`

File: `lava-api-go/internal/handlers/v1/handlers.go`

`writeProviderError` attrs (L172-193):

```
AttrFeature:   "provider"
AttrOperation: c.FullPath()          // route template, e.g. "/v1/
        {provider}/search"
AttrEndpoint:  c.FullPath()
AttrTrackerID: currentProviderID(c)  // e.g. "rutracker"
AttrRequestID: requestID(c)          // X-Request-ID header value
```

`c.FullPath()` returns the Gin route pattern, not the real URL — no query string, no credential. `currentProviderID(c)` returns a tracker-id string. `requestID(c)` returns the `X-Request-ID` header (a correlation UUID, not a credential). CLEAN.

3.3 `internal/handlers/v1/search.go`

File: `lava-api-go/internal/handlers/v1/search.go`

`GetMultiSearch` unknown-provider `RecordNonFatal` attrs (L162-167):

```
AttrFeature:      "search"
AttrOperation:    "multi_search_get_provider"
AttrEndpoint:     c.FullPath()
AttrTrackerID:    pid              // provider id string
AttrErrorClass:   "provider_not_found"
AttrErrorMessage: err.Error()     // "provider 'X' not found"
```

`err.Error()` is a `fmt.Errorf("provider '%s' not found", pid)` — does not contain credentials. CLEAN.

SSE streaming failures (L205-222): these use `// no-telemetry:` opt-out because the SSE event stream IS the telemetry surface — no double-reporting. CLEAN (explicit opt-out).

3.4 `internal/middleware/firebase.go` — `FirebaseTelemetry` middleware

File: `lava-api-go/internal/middleware/firebase.go`

Panic recovery attrs (L55-61):

```
"http.method":   c.Request.Method      // GET / POST
"http.path":     c.FullPath()          // route pattern
"http.url":      c.Request.URL.RequestURI()   // ← SEE NOTE
"event.type":    "panic"
"event.elapsed": duration
```

5xx attrs (L69-74):

```
"http.method":   c.Request.Method
"http.path":     c.FullPath()
```

```
"http.status":   status code
"event.elapsed": duration
```

NOTE on `"http.url"` in the panic path (L58):
`c.Request.URL.RequestURI()` returns the request URI including query string. For Go API search requests, the query string carries search parameters (`?q=term&page=N`). Auth is passed as the `Lava-Auth` HTTP header — it does NOT appear in the query string. The `sensitiveAttrPatterns` `"authorization"` pattern would redact an `Authorization` header-name attr key, but `"http.url"` does NOT match any sensitive pattern — so if a secret somehow appeared in the URL (e.g. via a future misuse), it would NOT be redacted by `redactIfSensitive`.

However: the Go API's router enforces that auth is always a header (Gin middleware extracts it from `c.GetHeader(lavaAuthFieldName)`). No Go handler places auth in the URL. This is an architectural property, not a redaction property. The note is recorded here for the operator's awareness — it is not a current leak but is the highest-risk point in the audit.

`clientFirebase.RecordNonFatal` (L82) passes `fields` which is the 5xx attr map above (no `"http.url"` — that only appears in the panic deferred block). 5xx fields contain only method, path, status, elapsed. CLEAN.

3.5 `internal/firebase/firebase.go` — the Go Firebase client

File: `lava-api-go/internal/firebase/firebase.go`

Current implementation: `adminClient.RecordNonFatal` (L151-159) is a structured-log forwarder (`slog.LevelError` with `fields` passed as-is). No Admin SDK wiring yet (comment at L124 notes wiring is in a follow-up commit). `noopClient.RecordNonFatal` (L114-119) logs to `slog.Warn` with all fields. In both cases the sink receives exactly what the middleware passes — no enrichment, no additional data extraction. CLEAN.

---

4. `RuTrackerNetworkApi.kt` — telemetry reference

File: `core/tracker/rutracker/src/main/kotlin/lava/tracker/rutracker/impl/RuTrackerNetworkApi.kt`

The grep at L71 shows a comment reference only (`// + recordWarning telemetry (§6.AC)`). No `recordWarning` implementation was found at that line — the actual call site is implemented higher in the call stack (in

`ApiBackedTrackerClient`, which wraps the HTTP layer and constructs `ApiHttpException`). The `RuTrackerNetworkApi` itself does not directly call `analytics.*`. CLEAN by delegation.

---

5. Three strongest file:line citations

1. **`ApiBackedTrackerClient.kt:126–137`** — Auth credentials are set as HTTP headers via `withAuth()`, never in URL query params. This is the primary architectural protection that makes all URL-stripping and query-parameter analysis secondary.

2. **`ApiBackedTrackerClient.kt:273 / L276`** — `ApiHttpException.responseSnippet = redactAndTruncate(resp.body?.string())` and `requestUrl = stripQueryForTelemetry(url)` — both applied before the exception is constructed and before any telemetry call sees these fields.

3. **`ApiKeyClient.kt:53–58 vs L66–87`** — the key value is obtained at L57 (`cursor.getString(keyIdx)`) only in the success branch; both exception catch branches at L60-88 record only `e.message` (OS exception message), never the key value.

---

6. Overall verdict

CLEAN.

No credential, token, cookie, key, pepper, handoff-key, or Auth-Token value reaches Firebase Crashlytics, structured logs, or the observability webhook through any of the audited telemetry paths. The layered defence is:

1. **Architectural**: auth is always a header, never a URL param (primary protection).
2. **Structural**: `ApiHttpException` fields are set via `stripQueryForTelemetry` and `redactAndTruncate` before reaching any call site.
3. **Call-site discipline**: every `recordWarning` / `recordNonFatal` call passes only structured diagnostic data (provider IDs, operation names, error class strings, exception messages from OS-level exceptions). No call site passes a key value.
4. **Go webhook redaction**: `redactIfSensitive` applied to all attrs before webhook POST.

---

7. Notable (not a leak, documented for operator awareness)

7.1 `ApiHttpException.message` contains search-term query string

The throwable's `message` field (`"API request failed: HTTP ${resp.code} for $url"`, `ApiBackedTrackerClient.kt:279/293/314`) contains the **raw URL** including query string. URLs follow the pattern `{apiBaseUrl}/v1/{trackerId}/{op}?query=SEARCH_TERM&page=N&...`.

When `recordNonFatal(cause, context)` is called in `SearchResultViewModel.kt:211`, Firebase Crashlytics captures this message via `recordException(throwable)` → `FirebaseCrashlytics.getInstance().recordException(throwable)`. The message reaches the Crashlytics dashboard.

This is NOT a credential leak. Query parameters contain search terms (user's search query, page number, category filter), not auth values. Auth is a header. However, if operator policy considers user search queries sensitive, `ApiHttpException` could be refactored to omit the query string from `message` as well (currently only `requestUrl` is stripped, not `message`). This is a privacy consideration, not a §6.H violation.

7.2 `"http.url"` in Go panic telemetry middleware

`FirebaseTelemetry` at `internal/middleware/firebase.go:58` passes `c.Request.URL.RequestURI()` (full URI including query string) as `"http.url"` in the panic-recovery attrs. The `redactIfSensitive` check uses attr KEY matching; `"http.url"` does not match any sensitive pattern, so the VALUE would not be redacted if it contained a credential.

This is NOT a current leak because Go router architecture ensures auth is always a header (`c.GetHeader(lavaAuthFieldName)`), never a query parameter. The Go API's URL patterns do not include auth params. However, the key `"http.url"` is the audit's highest-risk attr — any future handler that adds auth-bearing query params would bypass redaction. The operator should be aware of this.

Recommended hardening (optional): strip the query string from `"http.url"` in the panic middleware, e.g.:

```
"http.url": c.Request.URL.Path,  // path only, no query
```

---

8. Audit coverage

Surface	File(s) read	Verdict
`ApiHttpException` construction + `stripQueryForTelemetry` + `redactAndTruncate`	`ApiBackedTrackerClient.kt` (366 lines)	CLEAN
`recordProviderFailure` context	`SearchResultViewModel.kt` (561 lines)	CLEAN
`ApiKeyClient.recordWarning` (×2)	`ApiKeyClient.kt` (97 lines)	CLEAN
`OnboardingViewModel.recordNonFatal/recordWarning` (×6 sites)	`OnboardingViewModel.kt` (912 lines)	CLEAN
`ApiEngineService.recordNonFatal` (×1 site)	`ApiEngineService.kt` (320 lines)	CLEAN
`FirebaseAnalyticsTracker` sink	`FirebaseAnalyticsTracker.kt` (130 lines)	CLEAN (sink only)
`sensitiveAttrPatterns` + `redactIfSensitive` + `webhookForward`	`nonfatal.go` (305 lines)	CLEAN
`writeProviderError` attrs	`handlers.go` (194 lines)	CLEAN
`GetMultiSearch` RecordNonFatal attrs	`search.go` (282 lines)	CLEAN
`FirebaseTelemetry` middleware	`middleware/firebase.go` (124 lines)	CLEAN (see §7.2)
Firebase client (no-op + admin)	`firebase/firebase.go` (177 lines)	CLEAN
`RuTrackerNetworkApi`	reference only — delegates to `ApiBackedTrackerClient`	CLEAN